

Supplementary Material for LLM Agents Can See Code Repositories

1 Additional Benchmark Results on GPT-5-mini

To complement the SWE-bench Verified results reported in the main paper, we include an additional benchmark table for GPT-5-mini in the same effectiveness-efficiency format. The table covers two external evaluations that are directly relevant to reviewer requests for broader benchmark coverage: SWE-Rebench Leaderboard and SWE-Sharp-Bench. For SWE-Rebench Leaderboard, we report results on all 57 instances introduced in the 2026.02 release, which correspond to newly created issues spanning 41 different code repositories; under the text-only setting, the agent resolves 24 out of 57 instances. For SWE-Sharp-Bench, we currently report the text-only result and keep the multimodal setting for the revised version.

One implementation detail is worth clarifying when interpreting cross-language generalization. The current released artifact is Python-specific at the graph-construction stage: it builds repository graphs by parsing `.py` files with Python AST analysis. For our SWE-Sharp-Bench evaluation, we extended this stage to C# by replacing the Python-specific frontend with a C# static-analysis frontend while keeping the overall SeeRepo framework unchanged. Concretely, we scanned `.cs` files and constructed the same heterogeneous graph schema with directory/file/class/method nodes and four relation types: **contains** from filesystem and member nesting, **imports** from **using** directives and namespace-level dependencies, **inherits** from base-type declarations, and **invokes** from method invocation expressions resolved within the repository. The graph visualization interface, query mechanism, and agent interaction protocol were kept the same, with only the node naming convention adapted from Python-style `path/to/file.py:Class.method` identifiers to the corresponding C# file/class/method identifiers. The SWE-Sharp-Bench results based on the C# frontend are already included below. The multimodal setting achieves comparable Pass@1 performance while reducing average interaction turns, token usage, and cost, suggesting that the efficiency benefits of SeeRepo generalize beyond Python repositories.

Table 1: Additional benchmark results on GPT-5-mini.

Benchmark	Setting	Effectiveness	Efficiency			
		Pass@1	API Calls	Input Tokens	Output Tokens	Cost (\$)
SWE-Rebench Leaderboard	Text	42.1%	26	388,833.96	6,550.79	0.052
	Multimodal	43.9% (+1.8)	23 (-12%)	352,511.42 (-9.3%)	5,817.44 (-11.2%)	0.047 (-9.6%)
SWE-Sharp-Bench	Text	26.0%	23	301,640.62	5,809.82	0.041
	Multimodal	25.3% (-0.7)	21 (-8.7%)	277,159.48 (-8.1%)	5,507.36 (-5.2%)	0.038 (-7.8%)

2 RQ3 and RQ4 Experiments on Doubao-Seed-2.0-Lite

This section provides a supplementary numerical analysis for Doubao-Seed-2.0-Lite in the same format as the main-paper RQ3 and RQ4 tables. Under RQ3, the graph layout remains the most efficient default design, while text increases token usage despite a smaller Pass@1 gain and nested/tabular trade additional cost for alternative visual organization. Under RQ4, localization remains the most effective stage for visualization, while moving visualization to repair or patch validation leads to a clear reduction in Pass@1.

Table 2 reports the numerical results for layout and depth settings. In the layout comparison, text yields only a smaller Pass@1 improvement while increasing input tokens beyond the baseline. By contrast, graph and tabular improve Pass@1 by 1.0 point, while nested remains on par with the baseline; graph still remains the most efficient configuration. In the depth comparison, fixed-depth settings can recover additional effectiveness, with Depth = 1 reaching the highest Pass@1, whereas the adaptive strategy remains the strongest efficiency-oriented operating point.

Table 2: Supplementary RQ3 analysis on Doubao-Seed-2.0-Lite.

Setting	Pass@1	API Calls	Input Tokens	Output Tokens	Cost (\$)
Mini-SWE-Agent	51.0%	22	185,311	5,383	0.0173
Layout					
+ Text	51.6% (+0.6)	22 (+0%)	194,985 (+5.2%)	5,973 (+10.9%)	0.0190 (+9.8%)
+ Graph	52.0% (+1.0)	21 (-4%)	176,463 (-4.8%)	4,958 (-7.9%)	0.0162 (-6.0%)
+ Nested	51.0% (+0.0)	21 (-4%)	181,402 (-2.1%)	5,314 (-1.3%)	0.0169 (-2.3%)
+ Tabular	52.0% (+1.0)	21 (-4%)	183,976 (-0.7%)	5,377 (-0.1%)	0.0171 (-1.2%)
Depth					
Depth = 1	53.8% (+2.8)	22 (+0%)	187,164 (+1.0%)	5,319 (-1.2%)	0.0172 (-0.6%)
Depth = 2	52.0% (+1.0)	22 (+0%)	184,940 (-0.2%)	5,434 (+0.9%)	0.0174 (+0.6%)
Depth = 3	52.4% (+1.4)	22 (+0%)	184,755 (-0.3%)	5,544 (+3.0%)	0.0175 (+1.2%)
Depth = 4	51.0% (+0.0)	22 (+0%)	186,794 (+0.8%)	5,789 (+7.5%)	0.0180 (+4.2%)
SeeRepo (Adaptive)	52.0% (+1.0)	21 (-4%)	176,463 (-4.8%)	4,958 (-7.9%)	0.0162 (-6.0%)

Table 3 reports the stage-specific results. The same stage sensitivity observed for GPT-5-mini is preserved here: localization yields the strongest overall trade-off, repair causes the largest degradation in effectiveness, and patch validation partially recovers but remains below localization.

Table 3: Supplementary RQ4 analysis on Doubao-Seed-2.0-Lite.

Stage	Pass@1	API Calls	Input Tokens	Output Tokens	Cost (\$)
Mini-SWE-Agent	51.0%	22	185,311	5,383	0.0173
Localization	52.0% (+1.0)	21 (-4%)	176,463 (-4.8%)	4,958 (-7.9%)	0.0162 (-6.0%)
Repair	46.6% (-4.4)	22 (+0%)	189,742 (+2.4%)	5,248 (-2.5%)	0.0173 (-0.1%)
Patch Validation	48.2% (-2.8)	22 (+0%)	191,362 (+3.3%)	5,288 (-1.8%)	0.0174 (+0.7%)

Taken together, the supplementary numerical analysis shows that Doubao-Seed-2.0-Lite follows the same broad ablation pattern as GPT-5-mini. The graph layout remains the most efficient layout choice, adaptive depth remains the strongest overall efficiency-oriented configuration, and localization remains the most appropriate stage for incorporating repository visualization.

3 Supplementary SWE-QA Results on GPT-5-mini

This section provides additional evidence beyond issue resolution by evaluating SeeRepo on SWE-QA, a repository-level code question answering benchmark. All experimental settings follow the original SWE-QA paper. Following the official SWE-QA evaluation protocol, we report the Overall Score (0–100) together with efficiency statistics. The overall trend remains consistent with the main paper: multimodal structural context preserves answer quality while reducing exploration cost. We further observe that the efficiency gain is even more pronounced on repository-level code question answering than on issue resolution. A likely reason is that SWE-QA mainly requires locating the relevant code, rather than completing the full issue-resolution pipeline. As a result, graph-based structural grounding can more directly shorten repository exploration, leading to a larger reduction in API calls and token usage.

Table 4: Supplementary SWE-QA results on GPT-5-mini.

Setting	Overall Score	API Calls	Input Tokens	Output Tokens	Cost (\$)
Text	66.8	14	66,369	5,003	0.0107
Multimodal	67.2 (+0.4)	9 (-35.7%)	50,038 (-24.6%)	4,254 (-15.0%)	0.0079 (-26.2%)

4 Statistical analysis of SeeRepo among different Multimodal Models

This section presents an additional trajectory-level analysis for RQ2, with the goal of clarifying how SeeRepo affects different multimodal models in distinct ways. The analysis focuses on three representative models: GPT-5-mini, Kimi K2.5, and Doubao-Seed-2.0-Lite. We additionally observe that GPT-5.1 exhibits a trend similar to GPT-5-mini, and therefore omit it here for brevity. Across the supplementary figures, a consistent pattern emerges: GPT-5-mini primarily uses the graph as a lightweight localization aid, with a strong preference for `imports` and shallow `contains` queries; Kimi K2.5 uses the graph more deeply as part of a structure-guided reasoning loop, frequently continuing from `contains` to `invokes` and occasionally to `inherits`; and Doubao-Seed-2.0-Lite exhibits an intermediate behavior, relying heavily on `contains`-based directory navigation while performing only limited deeper tracing. Quantitatively, among instances that actually invoke the graph tool, the average numbers of graph queries are 1.36 for GPT-5-mini, 2.86 for Kimi K2.5, and 1.55 for Doubao-Seed-2.0-Lite.

These differences provide a consistent explanation for the effectiveness-efficiency trade-offs reported in RQ2. GPT-5-mini benefits mainly through shorter and more targeted localization, which substantially reduces API calls and token cost while leaving overall Pass@1 nearly unchanged. Kimi K2.5 uses the graph as part of a richer structural reasoning process, which helps explain why its improvement is more visible in effectiveness than in efficiency. Doubao-Seed-2.0-Lite benefits mainly from directory- or module-level navigation assistance, leading to moderate gains in both dimensions. Although these observations should not be interpreted as establishing a strict causal mechanism, they provide a behaviorally grounded account that is directly supported by trajectory statistics, relation preferences, query depths, and node-granularity patterns. Detailed figure-level analyses are provided in the following subsections.

4.1 Graph Query Node Granularity

Figure 1 further distinguishes the models by the semantic granularity of the queried graph nodes. GPT-5-mini operates almost entirely at the directory and file levels, which is consistent with a coarse but efficient localization strategy. Kimi K2.5 more frequently queries classes and methods, especially together with `invokes` and `inherits`, indicating a substantially finer-grained use of the structural representation. Doubao-Seed-2.0-Lite again occupies an intermediate position, but remains much closer to directory-centric behavior. This observation provides an additional explanation for the heterogeneous RQ2 outcomes: the models are not merely consulting the same graph with different frequencies, but are operating on different semantic levels of the repository structure.

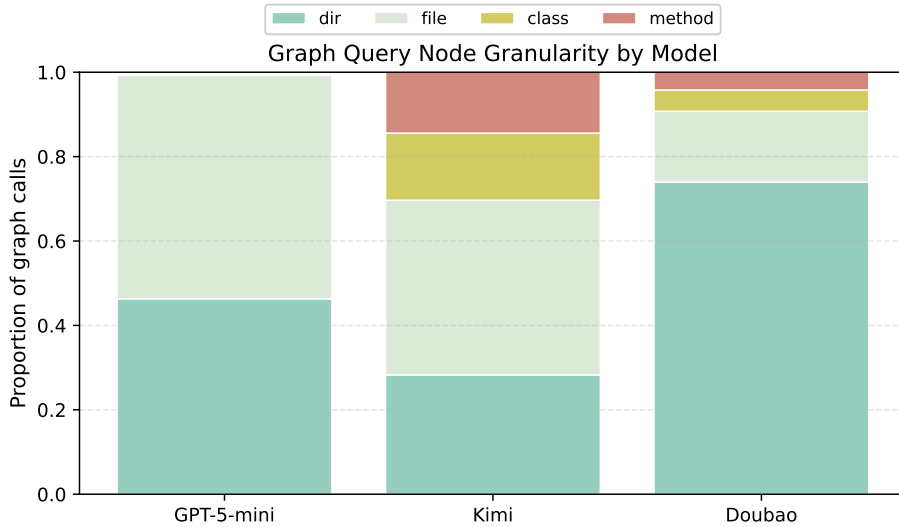


Figure 1: Granularity of queried nodes (directory, file, class, method) across models.

4.2 Graph Relation Preference

Figure 2 shows that the models differ not only in how often they use the graph, but also in which structural relations they choose to inspect. GPT-5-mini concentrates almost entirely on `imports` and `contains`, indicating a predominantly file-level or module-level localization strategy. Kimi K2.5 uses substantially more `invokes`, revealing a stronger tendency to inspect finer-grained execution structure at the function or method level. Doubao-Seed-2.0-Lite is dominated by `contains`, which is consistent with a directory-centric or module-centric navigation strategy. This figure therefore converts the previously qualitative distinction among the models into direct quantitative evidence.

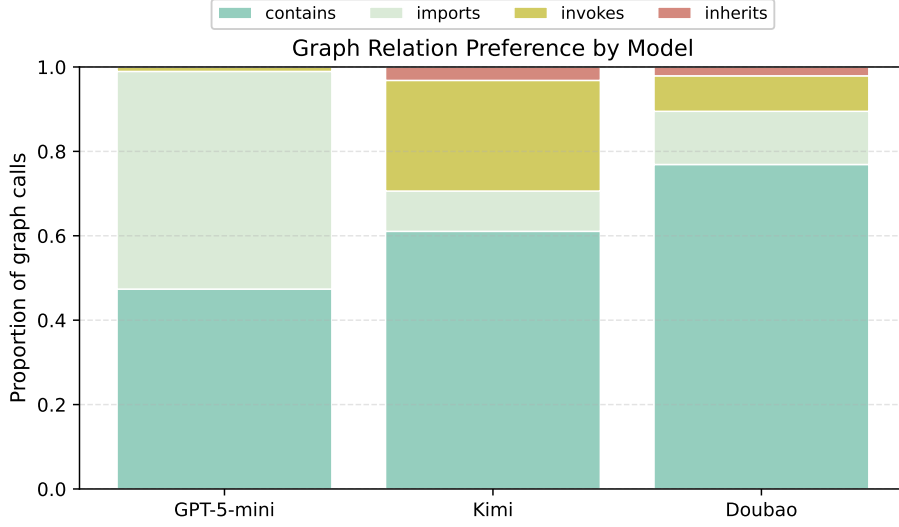


Figure 2: Distribution of graph relation types queried by different models.

4.3 Graph Query Depth

Figure 3 highlights another systematic distinction in graph usage. GPT-5-mini is highly conservative and concentrates on shallow settings, especially (1, 1), which is consistent with the use of the graph as a precise and inexpensive localization aid. Kimi K2.5 explores a much broader range of depth settings, reflecting a willingness to enlarge the structural search space when necessary. Doubao-Seed-2.0-Lite lies between these two behaviors, although it remains closer to shallow directory scanning than to deep drill-down. This depth difference helps explain the different RQ2 outcomes: GPT-5-mini obtains stronger efficiency gains because it uses the graph conservatively, whereas Kimi gains more in effectiveness because it invests more computation in structure-aware exploration.

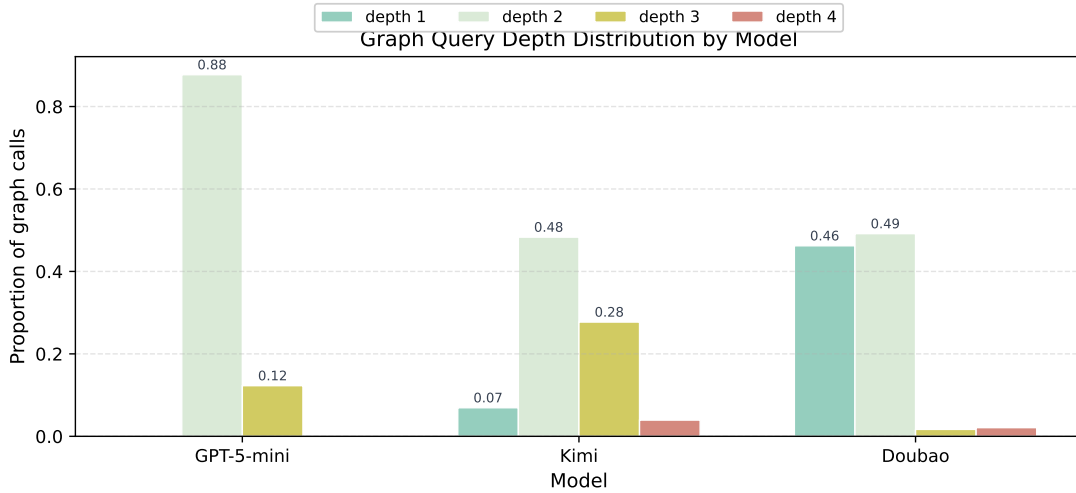


Figure 3: Heatmap of graph-query depth choices (up_depth, down_depth) across models.

4.4 Action Distribution Before and After the First Graph Query

Figure 4 provides a finer-grained view by separating actions before and after the first graph query in graph-using trajectories. GPT-5-mini typically invokes the graph very early, often after an initial listing-heavy phase, and then transitions quickly into code reading, editing, and testing. This pattern suggests that the graph serves as an early structural pivot that accelerates the move from coarse navigation to concrete repair actions. Kimi K2.5 differs in that substantial code reading is already present before the first graph query, and remains prominent afterwards. This indicates that graph usage is embedded within a longer reasoning loop rather than functioning as a one-shot replacement for search. Doubao-Seed-2.0-Lite resembles GPT-5-mini in that graph usage often appears early, but its post-graph trajectories retain a stronger editing component, suggesting a directory-oriented navigation pattern followed by relatively direct repair attempts.

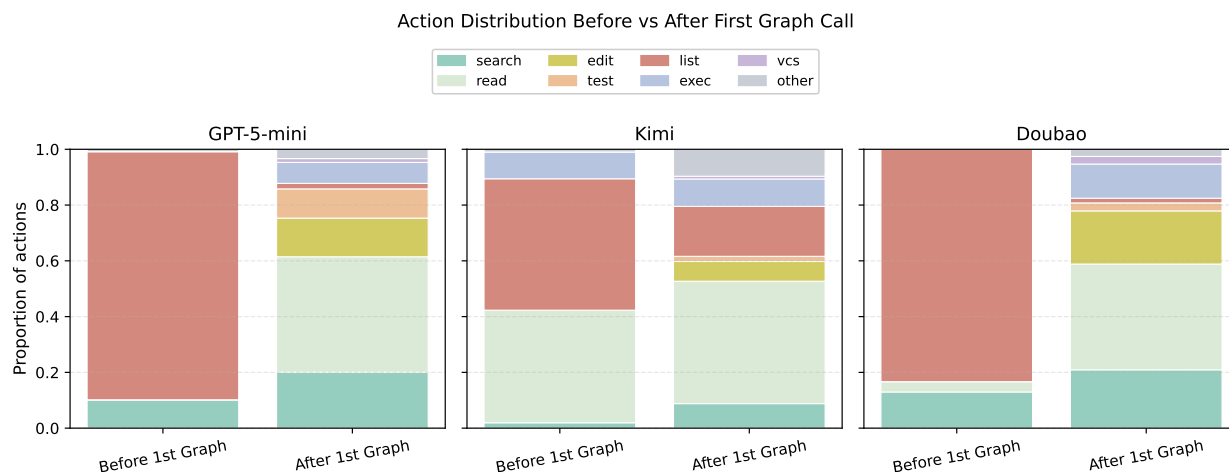


Figure 4: Distribution of assistant actions before and after the first graph query in graph-using trajectories.